

UM Bicycle & Scooter Rental Management System

Team Name: Team 01

Team Members:

Original Pair	Student ID	Student Name	Role
09	dc325381	Hoi Kai Cheng	Leader (Database Schema Design, Admin/Staff Module & Deployment Pipelines)
09	dc325107	Ho Weng Hong	Member (Backend Architecture & Core API Logic)
07	dc325182	Che Chi Hin	Member (Frontend Lead, UI/UX Design & Geofencing)
07	dc325420	Chang I Wai	Member (Responsive Web Design (RWD) CSS & Rate Limiting logic)
14	dc325095	Chung Hou Sam	Member (Public Landing Pages & Comprehensive Documentation)
14	dc325837	Lei Man Lam	Member (i18n Framework & Backend Security)

Date: May 7, 2026

Table of Contents

UM Bicycle & Scooter Rental Management System.....	1
1. Project Abstract.....	3
2. Project Context.....	4
3. Project Goals.....	5
4. Project Services and Data Requirements.....	6
4.1 Core Service Requirements.....	6
4.2 Data Requirements (Entities).....	7
5. Project Tasks and Tools.....	9
5.1 Technical Tasks by Layer.....	9
5.2 Development and Deployment Tooling.....	9
6. Project Challenges.....	11
7. Project Deliverables.....	12
8. Project Division of Work.....	12
9. Project Contributions.....	13
10. Project Criteria of Success.....	13
11. Project Evaluation.....	14
12. Project Learning.....	14
12.1 Individual Learning (All Members).....	14
12.2 Team Learning.....	15
13. Project Test Cases (Functional Requirements).....	16
14. Project Demo Walkthrough (ScreenShots).....	18
14.1 Public and Authentication Flow.....	18
14.2 Student User Experience.....	18
14.3 Staff and Admin Operations.....	19
14.4 Responsive Web Design (RWD).....	19
15. Project Citations and References.....	21
16. Declaration of AI Usage.....	22
17. Installation & Deployment Instructions.....	23
17.1 Local Development and Testing (XAMPP).....	23
17.2 Live Deployment (InfinityFree).....	23
17.3 Demo Reset Procedure.....	23
17.4 Container and Cloud Readiness.....	23
18. Incomplete Items (Not completed by deadline).....	24
19. Remarks and Academic Integrity Statement.....	24

1. Project Abstract

The UM Bicycle & Scooter Rental Management System is a full-stack web application developed to improve campus mobility for the University of Macau (UM) community. The platform provides a centralized rental ecosystem with three role-specific portals: Student, Staff, and Administrator. Students can discover vehicles on an interactive Leaflet map, start and end rentals through guided modal workflows, and manage wallet top-ups and payment deductions. Staff users can monitor fleet conditions and support daily operations, while administrators are granted privileged controls for user governance, abnormal-order handling, and operational overrides.

The system is implemented using a LAMP/XAMPP architecture with PHP, MySQL, and Apache, and a frontend built with HTML, CSS, and JavaScript under Responsive Web Design (RWD) principles. This ensures consistent usability across desktop and mobile devices while preserving a modern UM-branded visual identity. Security and reliability are reinforced through role-based access control (RBAC), protected endpoint boundaries, sensitive-file access restrictions, and rate-limiting safeguards to reduce API abuse risk.

A major project achievement is the integration of a complete internationalization (i18n) framework across both public and authenticated pages, supporting English, Simplified Chinese (zh-CN), and Traditional Chinese (zh-TW). The language switch workflow is designed for practical real-time use and consistent interface coverage. To support grading and reproducibility, the team also implemented separate seed/reset workflows for demo and development scenarios, reducing data inconsistency across team environments.

For environment usage, the team uses a local XAMPP server for website testing and debugging. For actual deployment, the website runs on InfinityFree, with its built-in phpMyAdmin managing the MySQL database used by the system. The project is also containerized with Docker and prepared for future Google Cloud Platform integration (Cloud Run + Cloud SQL). Overall, the project delivers a secure, scalable, and user-oriented mobility solution tailored to UM's transportation needs.

2. Project Context

Full-Stack Web Application with RWD and Deployment Readiness

The UM Bicycle & Scooter Rental Management System is developed as a centralized full-stack web platform using a standard LAMP/XAMPP architecture (PHP, MySQL, and Apache). The system supports end-to-end rental operations for multiple user roles, including students, staff, and administrators, within a unified service environment.

On the frontend, the application is implemented with HTML, CSS, and JavaScript, with strict adherence to Responsive Web Design (RWD) principles to ensure usability across desktop and mobile devices. The interface design follows the University of Macau's visual identity through a purple-and-white color scheme and a modern, user-centered layout.

The backend is responsible for business logic, role-based access control, rental workflow handling, wallet transactions, and administrative operations. Data persistence and query processing are managed through MySQL with structured relational schemas for users, vehicles, orders, and financial records.

Source Code Repository: <https://github.com/HOWENGHONG/um-rental-system>

Deployed URL: <https://cisc3003team01.is-great.net>

Deployment Status:

The team uses a local XAMPP server for development, testing, and debugging. The live demo system is deployed on InfinityFree, and its built-in phpMyAdmin is used to manage the MySQL database (schema import, data maintenance, and verification). The project is also containerized with Docker and remains prepared for future Google Cloud Platform deployment (Cloud Run + Cloud SQL).

3. Project Goals

The project is guided by the following core goals:

- **Improve Campus Mobility:** Provide a reliable, digital-first transportation solution that enables UM students to rent bicycles and scooters efficiently for short-distance campus travel.
- **Enforce Strict Access Control:** Implement clear role boundaries among Students (service consumers), Staff (daily operators), and Administrators (system managers), with role-based permissions to protect data integrity and operational security.
- **Deliver Seamless Localization:** Achieve full user interface localization across English, Simplified Chinese (zh-CN), and Traditional Chinese (zh-TW) to support UM's multilingual user base and improve accessibility.
- **Ensure Operational Readiness:** Maintain separate runtime data profiles for different use cases, including production-like operation and grading/demo scenarios (e.g., *seed_demo_data.sql* versus *basic_seed.sql*), to improve reliability, reproducibility, and evaluation clarity.

4. Project Services and Data Requirements

4.1 Core Service Requirements

The UM Bicycle & Scooter Rental Management System provides role-specific services under a strict RBAC model to ensure secure and efficient operations.

For **student users**, the platform supports:

- Secure account authentication with student-role enforcement
- Personal profile management and digital wallet top-up/usage
- Real-time vehicle discovery through an interactive campus map
- Modal-based rental workflows (start rent / end rent)
- Rental order history and status tracking

For staff and administrators, the system provides:

- Role-based portal authentication and protected management access
- Student account monitoring and management
- Dynamic fleet and station inventory control
- Abnormal order handling and operational intervention

Force-close actions for unresolved or exceptional rental sessions

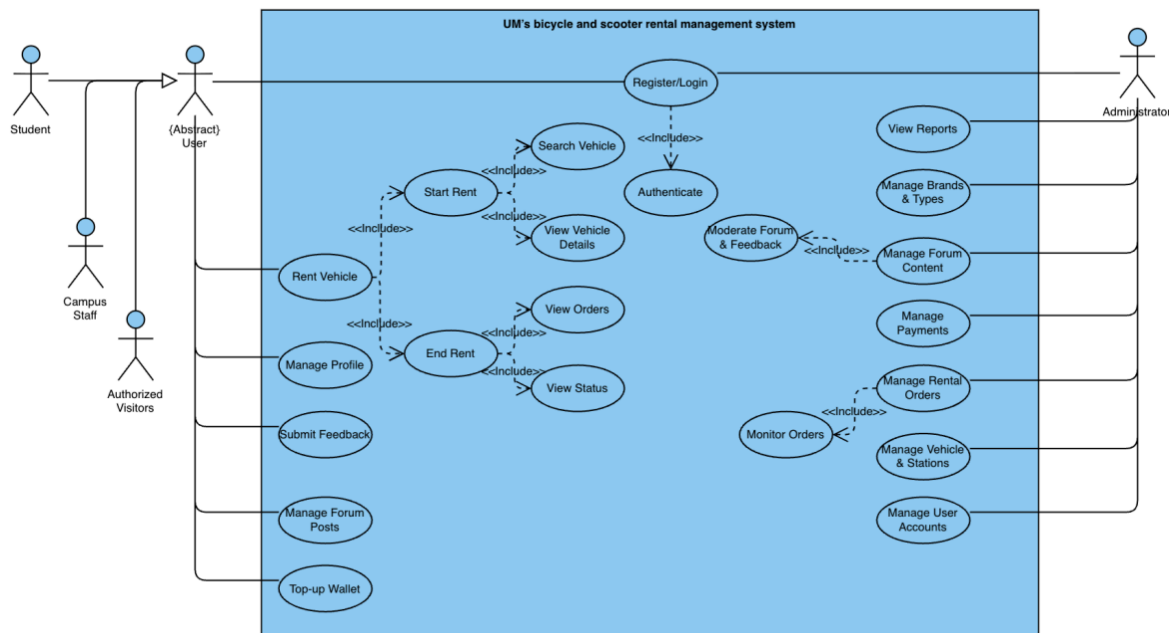


Figure 1 (Use Case Diagram) illustrates the complete functional scope and RBAC boundaries

among Students, Staff, and Administrators, including shared and restricted system capabilities.

4.2 Data Requirements (Entities)

To support functional correctness, auditability, and scalability, the system defines the following core data entities:

- **Users:** Campus ID, full name, role (*student / staff / admin*), password hash, wallet balance
- **Vehicles & Brands:** Unique vehicle ID (e.g., *BK-0001*), type, brand ID, real-time status (*Available, Active, In-Use, Maintenance*)
- **Rental Stations:** Location ID, station name, coordinates (latitude/longitude strictly bounded to UM campus), capacity
- **Rental Orders:** Order number, user ID, vehicle ID, rental start/end timestamps, calculated fee
- **Financial & Security Data:** Wallet transaction records (audit trail) and rate-limit metadata (API spam/abuse protection)

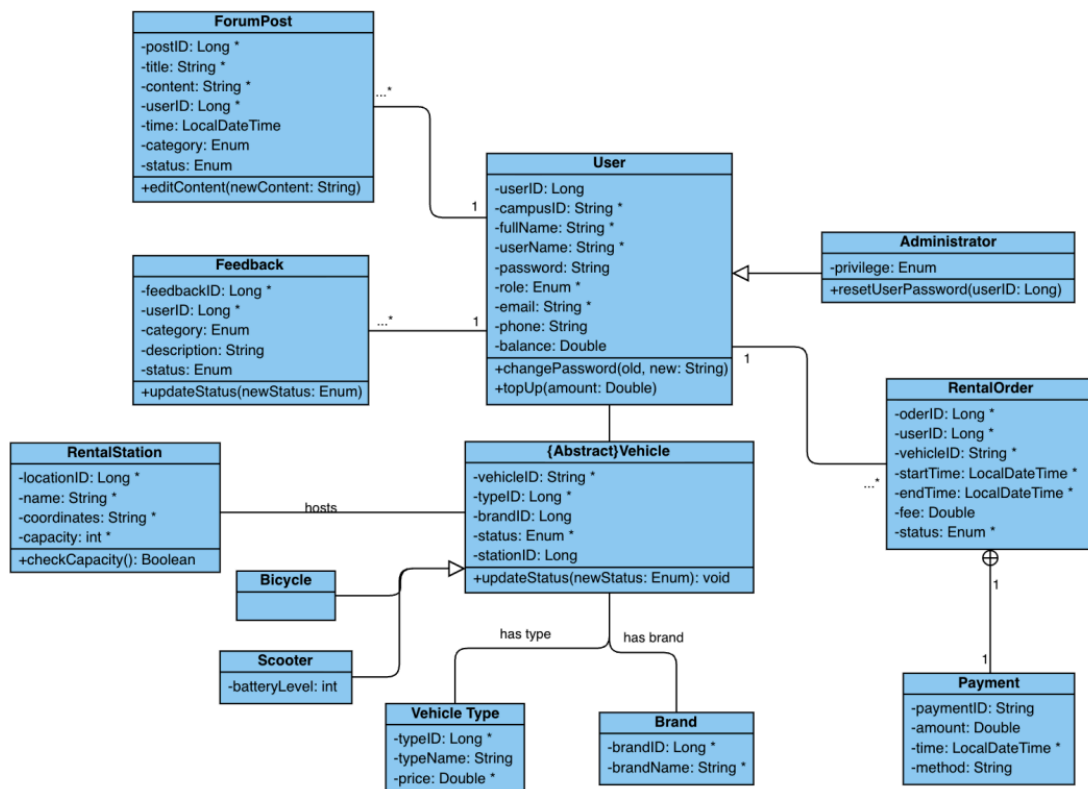


Figure 2 (ER Diagram) defines the key entities, attributes, and relationships used to maintain data normalization, referential integrity, and consistency across rental, financial, and

administrative workflows.

5. Project Tasks and Tools

The UM Bicycle & Scooter Rental Management System is implemented using a layered full-stack architecture designed for maintainability, modular development, and deployment flexibility.

5.1 Technical Tasks by Layer

- **Frontend Layer**
 - Built with HTML5, CSS3, and Vanilla JavaScript to deliver a responsive and mobile-friendly user experience. The interface supports role-specific workflows and integrates Leaflet.js for interactive campus map visualization and real-time vehicle discovery.
- **Backend Layer**
 - Developed in PHP 8+ to handle core business logic, session control, RBAC enforcement, and REST-style API endpoints for rental operations, wallet processing, and administrative management.
- **Data Layer**
 - Implemented using MySQL for persistent relational storage. The data model supports users, vehicles, stations, rental orders, wallet transactions, and auditability requirements. Controlled reset scripts (e.g., `seed_demo_data.sql`) are used to ensure reproducible demo/grading environments.

5.2 Development and Deployment Tooling

- XAMPP for local integrated development (Apache + PHP + MySQL)
- Git & GitHub for version control, collaboration, and change tracking
- Docker for containerization and environment consistency
- Google Cloud Platform SDK for cloud deployment preparation and future production rollout
- Figure 3 (System Architecture Diagram) presents the high-level architecture and integration flow, showing how the PHP backend securely bridges the responsive frontend and the MySQL database while supporting external service extensions and deployment portability.



Figure 3 (System Architecture Diagram) presents the high-level architecture and integration flow, showing how the PHP backend securely bridges the responsive frontend and the MySQL database while supporting external service extensions and deployment portability.

6. Project Challenges

- **API Response Corruption**
 - Some rate-limiting routines failed when supporting database tables were unavailable, causing PHP warnings/notices to leak into API outputs. This corrupted expected JSON responses and broke frontend parsing.
 - Resolution: Added defensive *try/catch* handling and fail-safe logic around rate-limit checks to guarantee clean JSON responses.
- **Geospatial Data Errors**
 - Early map rendering issues were caused by incorrect latitude/longitude ordering, resulting in misplaced markers.
 - Resolution: Introduced coordinate normalization and enforced strict UM campus map constraints via boundary controls (e.g., *maxBounds*) to prevent out-of-scope display and interactions.
- **Database State Management Instability**
 - Re-seeding workflows occasionally disrupted foreign key relationships (e.g., brand references for bicycles/scooters), reducing environment reproducibility.
 - Resolution: Separated demo and production seed workflows, and implemented a dedicated *reset_demo_state.sql* script for controlled, safe state reset.
- **UI and Backend State Synchronization**
 - A status mismatch occurred when staff-side operations marked vehicles as *active* while student-side renting logic only accepted *available*.
 - Resolution: Added backend compatibility mapping so operational states remain internally meaningful while still satisfying student rental eligibility checks.

7. Project Deliverables

The project deliverables include implementation artifacts, deployment assets, and supporting technical documentation.

- **Application Source Code**
 - Structured and maintainable PHP, JavaScript, and CSS codebase implementing core rental, management, and localization functions.
- **Database Scripts and Setup Assets**
 - Environment setup and maintenance files such as *setup_database.bat*, migration scripts, and state reset scripts for demo/grading reproducibility.
- **Technical Documentation**
 - *API_OVERVIEW.md* (API design and endpoint behavior)
 - *SECURITY_POLICY.md* (security controls and access policies)
 - *TEST_CHECKLIST.md* (functional verification coverage)
 - *DEPLOY_GCP.md* (cloud deployment procedures)
- **Cloud/Container Artifacts**
 - **Dockerfile** and **.dockerignore** for containerized deployment workflow and cloud-readiness preparation.

8. Project Division of Work

The project was collaboratively developed by six members and organized into three functional pairs to improve ownership clarity and delivery efficiency.

- **Ho Weng Hong & Hoi Kai Cheng**
 - System architecture leadership, database schema design, backend modules for Admin/Staff operations, deployment pipeline setup (Docker/GCP), and API development for rental and wallet workflows.
- **Che Chi Hin & Chang I Wai**
 - Front-end implementation, dashboard UI/UX design, Leaflet-based campus map integration, responsive web design (RWD) CSS, and rate-limiting logic for endpoint protection.
- **Chung Hou Sam & Lei Man Lam**
 - Internationalization (i18n) framework implementation, public landing page development (*home.php* and related pages), comprehensive technical documentation, and backend security hardening.

Task allocation was role-based but iterative, with cross-review and integration support among pairs during testing and release preparation.

9. Project Contributions

Each team member made distinct, high-impact contributions to the final system:

- **Ho Weng Hong**
 - Resolved critical cross-role data synchronization issues and finalized the secure single-admin structure policy to improve governance and operational safety.
- **Hoi Kai Cheng**
 - Resolved critical HTTP 500 failures in wallet top-up flows by restructuring wallet_transactions handling and finalized core rental API logic for production stability.
- **Che Chi Hin**
 - Delivered a Klook-inspired redesign of the student dashboard, including a polished modal rental workflow and improved success/error interaction states.
- **Chang I Wai**
 - Integrated the Leaflet campus map with strict UM boundary controls and implemented robust rate-limiting mechanisms to protect backend APIs from abuse.
- **Lei Man Lam**
 - Implemented dynamic i18n switching across public and authenticated pages (via localStorage) and strengthened backend security controls.
- **Chung Hou Sam**
 - Led the redesign of public-facing landing pages and consolidated project documentation into a structured SDM-style knowledge hierarchy.

10. Project Criteria of Success

Project success is evaluated using functional, security, usability, and localization criteria aligned with the PRM scope.

- **Functional Success**
 - A student can complete the full rental lifecycle without manual intervention: account registration/login, wallet top-up, vehicle discovery via map, rental start, rental end, and correct fee calculation reflected in order records.
- **Security Success**
 - RBAC is correctly enforced so that non-admin users cannot access admin-only endpoints or management interfaces. Sensitive server assets (e.g., .env, runtime logs, protected configs) are not directly accessible via browser requests.
- **Usability Success**
 - The platform maintains consistent layout, readability, and interaction quality across major desktop and mobile viewport sizes under RWD standards.

- **Localization Success**

- Language switching supports complete translation coverage of public and dashboard interfaces (English, zh-CN, zh-TW), with correct real-time UI updates and no hard reload requirement.

11. Project Evaluation

The project meets the core requirements defined in the PRM and demonstrates strong implementation maturity for a course-level full-stack system. The platform now supports a complete role-based workflow for Students, Staff, and Administrators, with stable operation across authentication, rental lifecycle, wallet handling, and management functions.

Key strengths include strict RBAC enforcement, reproducible demo-state controls, and a polished user experience that exceeds baseline functional expectations. The team successfully integrated responsive design, multilingual i18n support (English, zh-CN, zh-TW), and defensive backend handling for API stability and operational consistency.

From an engineering perspective, major architecture choices—layered full-stack design, structured relational schema, controlled seed/reset workflows, and containerization—improve maintainability, testing repeatability, and deployment portability. The application is validated in local XAMPP environments and has also been publicly deployed for demonstration.

Live Deployment URL: <https://cisc3003team01.is-great.net/>

At this stage, the main remaining improvement is production hardening for long-term cloud operations (e.g., full Cloud Run/Cloud SQL migration workflow and extended automated deployment checks). Overall, the project delivers a secure, usable, and scalable campus mobility solution aligned with its original goals.

12. Project Learning

12.1 Individual Learning (All Members)

Pair 09

Hoi Kai Cheng (dc325381)

I learned the complexity of synchronizing cross-role system states and preserving data integrity in multi-user workflows. Resolving role-based data sync issues taught me to design clearer state contracts between student-facing and staff-facing modules, especially for rental and admin operations.

Ho Weng Hong (dc325107)

I learned that backend reliability depends heavily on defensive error handling and schema consistency. During wallet top-up debugging, I realized how a small mismatch in transaction-table structure can trigger cascading API failures (e.g., HTTP 500), which reinforced the importance of migration discipline and fault-tolerant API design.

Pair 07

Che Chi Hin (dc325182)

I learned that good UI/UX is not only about visual design but also about interaction timing, error feedback, and user confidence. Building the dashboard and modal rental flow showed me how micro-level frontend decisions directly impact task completion and user trust.

Chang I Wai (dc325420)

I learned the practical challenges of geospatial implementation and security-aware frontend engineering. Integrating Leaflet with strict UM campus boundaries and implementing rate limiting improved my understanding of balancing usability, data correctness, and abuse prevention.

Pair 14

Chung Hou Sam (dc325095)

I learned that documentation is a core engineering deliverable rather than an afterthought. Structuring public pages and project documents into a coherent SDM-style format improved team alignment, onboarding speed, and submission readiness.

Lei Man Lam (dc325837)

I learned how localization and security are deeply connected to overall product quality. Implementing i18n across public and authenticated pages demonstrated the need for consistent translation coverage, state persistence, and secure backend validation across multilingual flows.

12.2 Team Learning

As a team, we learned that early standardization of environment setup and database migration is critical for full-stack collaboration. Manual and inconsistent local table creation across members caused synchronization issues (including the wallet top-up bug), which highlighted the need for unified setup scripts, controlled seed/reset workflows, and stricter versioned migration practices from the beginning of development.

13. Project Test Cases (Functional Requirements)

Test Case 1: Student Rental Flow (Happy Path)

- **Test ID: TC-FUNC-001**
 - **Objective:** Verify complete student rental lifecycle under normal conditions
 - **Preconditions:**
 - Valid student account exists
 - Wallet has sufficient balance
 - At least one rentable vehicle is available on map/station view
 - **Test Steps:**
 - Log in as a student user
 - Open the student dashboard
 - Select a station/vehicle from the map interface
 - Click Rent to start rental
 - Confirm successful rental modal/checkmark
 - End rental through the dashboard workflow
 - **Expected Result:**
 - Rental order is created and marked active after start
 - Vehicle state transitions correctly
 - Wallet verification is performed before confirmation
 - On rental end, fee is calculated and persisted correctly
 - Order history reflects the completed transaction

Test Case 2: Insufficient Funds Protection

- **Test ID: TC-FUNC-002**
 - **Objective:** Ensure rental is blocked when wallet balance is insufficient
 - **Preconditions:**
 - Valid student account exists
 - Wallet balance is \$0 (or below required threshold)
 - Vehicle is otherwise available for rent
 - **Test Steps:**
 - Log in as the low-balance student account
 - Open dashboard and attempt to rent a scooter
 - Proceed in rental modal until wallet check is triggered
 - **Expected Result:**
 - System denies rental creation
 - Error prompt appears (e.g., “Insufficient wallet balance.”)
 - No active order is created

- Vehicle status remains unchanged

Test Case 3: Admin Access Control

- **Test ID: TC-SEC-001**
 - **Objective:** Verify RBAC prevents students from accessing admin pages
 - **Preconditions:**
 - User is authenticated as student role
 - Admin module route is known (e.g., admin/admin_dashboard.php)
 - **Test Steps:**
 - Log in as student
 - Manually enter admin URL in browser
 - Submit request to admin endpoint/page
 - **Expected Result:**
 - Access is denied by backend/session guard
 - User is redirected to a safe route (public home or student dashboard)
 - No admin content or privileged operations are exposed

14. Project Demo Walkthrough (ScreenShots)

This section demonstrates key user flows and system capabilities through selected screenshots.

14.1 Public and Authentication Flow

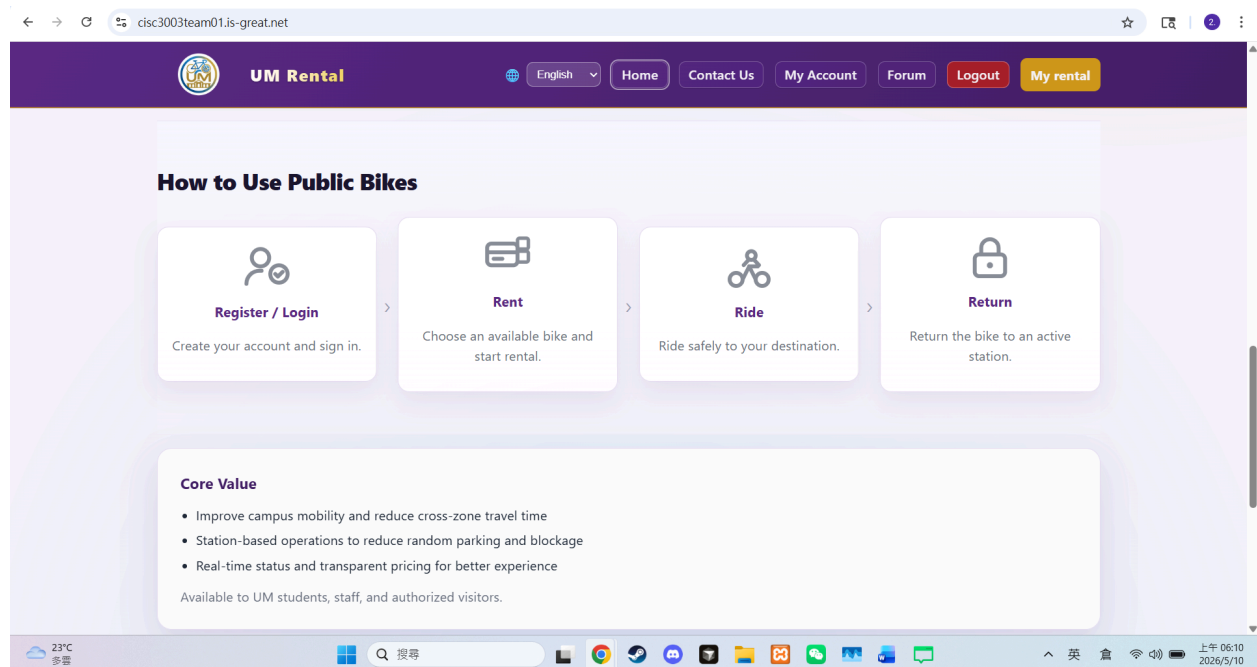


Figure 1. Home Landing Page

Shows the redesigned hero section, “How to Use Public Bikes” process cards, and UM-branded purple/white visual identity.

← → ↻ cisc3003team01.is-great.net/login.php ☆ 2

UM Rental English Home Services Why us How to use Contact Us [Login / Use Service](#)

Campus mobility, one account

Rent bicycles and e-scooters at UM stations—with transparent pricing and trip history.

UM Rental Login

Sign in using your Campus ID or email and password.

[Continue with Google](#)

or

Campus ID or Email

e.g. s1234567 or user@um.edu.mo

Password

Enter password

[Login](#)

New here? [Create an account](#)

[Back to Home](#)

← → ↻ cisc3003team01.is-great.net/register.php ☆ 2

UM Rental English Home Services Why us How to use Contact Us [Login / Use Service](#)

Create your campus profile

Register with your UM email, then sign in with Campus ID or email.

Create Account

Register with your email, then log in using Campus ID or email.

Campus ID

e.g. s1234567

Full Name

e.g. Chan Tai Man

Account type

Student

Email

e.g. user@um.edu.mo

Verification code

6-digit code [Send Code](#)

Phone (optional)

e.g. +853-0000-0000

Password

At least 8 chars with letters and numbers

Confirm Password

Re-enter your password

[Register](#)

Already have an account? [Login here](#)

[Back to Home](#)

Figure 2. Login and Registration

Shows the split-layout authentication interface and role-based registration options (student / teacher), with email verification and validation hints.

14.2 Student User Experience

UM Rental

English

Home

Contact Us

My Account

Forum

Logout

My rental

Find a ride

Pickup location

All stations

Central Teaching Building (E6)

Research Building (N21)

Science and Technology Building (E11)

Social Sciences and Humanities Building (E21)

Sports Complex (N8)

Student Activity Centre (E31)

University Mall (S8)

Wu Yee Sun Library (E2)

Pickup time

2026/05/10 上午 06:13

Drop-off time

2026/05/13 上午 06:13

Search

Central Teaching Building (E6)

See vehicles here

Research Building (N21)

See vehicles here

Science and Technology Building (E11)

See vehicles here

Social Sciences and Humanities Building (E21)

See vehicles here

Map & stations

See where bikes and e-scooters are available around campus.

My location

Refresh GPS

Set location manually

Could not access location. Allow location in your browser to see nearby stations.

Station	Parking spots	Bikes available	E-scooters available
Central Teaching Building (E6)	22	8	5
Research Building (N21)	26	4	7
Science and Technology Building (E11)	22	4	5
Social Sciences and Humanities Building (E21)	20	6	8
Sports Complex (N8)	24	10	7
Student Activity Centre (E31)	22	4	6
University Mall (S8)	20	6	5
Wu Yee Sun Library (E2)	24	6	6

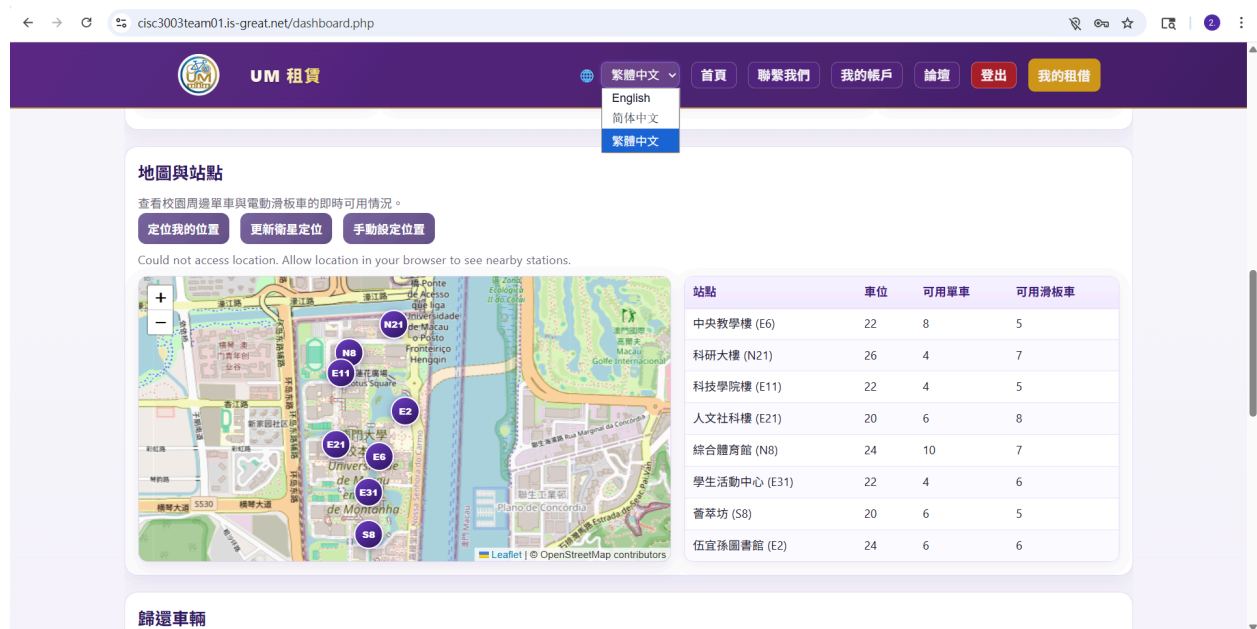
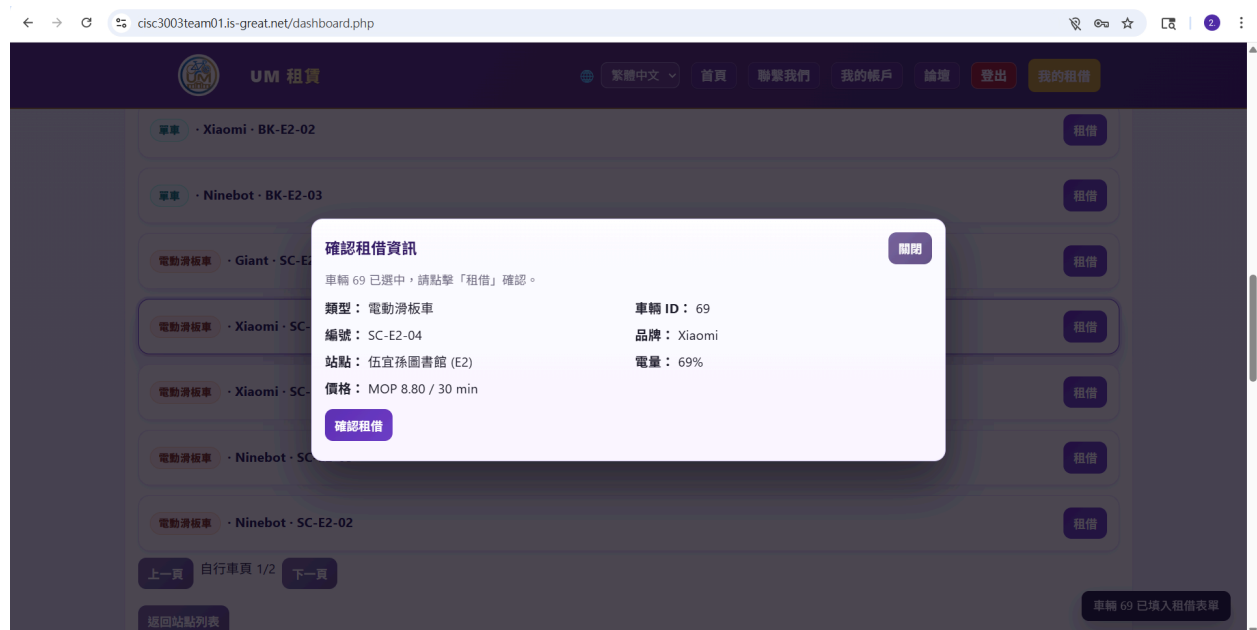


Figure 3. Student Dashboard

Shows the Leaflet-based UM campus map, station/vehicle filter controls, and i18n language switch result (Traditional Chinese view).



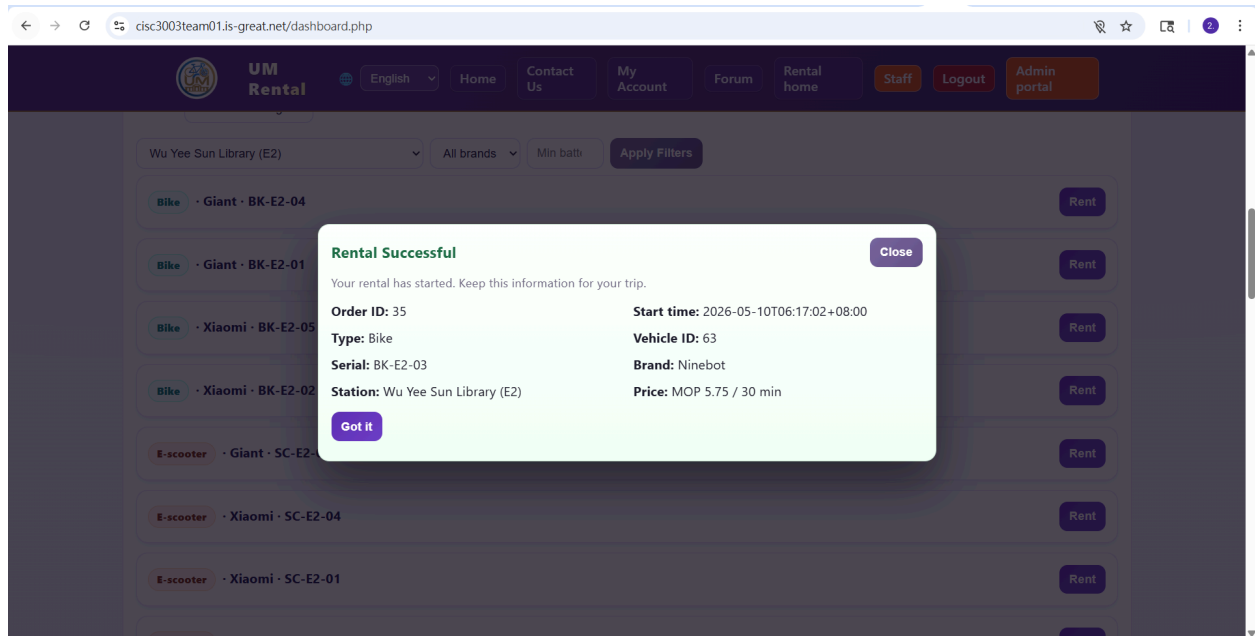


Figure 4. Rental Modal and Success State

Shows the rental confirmation modal (with dimmed background) and the successful rental confirmation state (green checkmark).

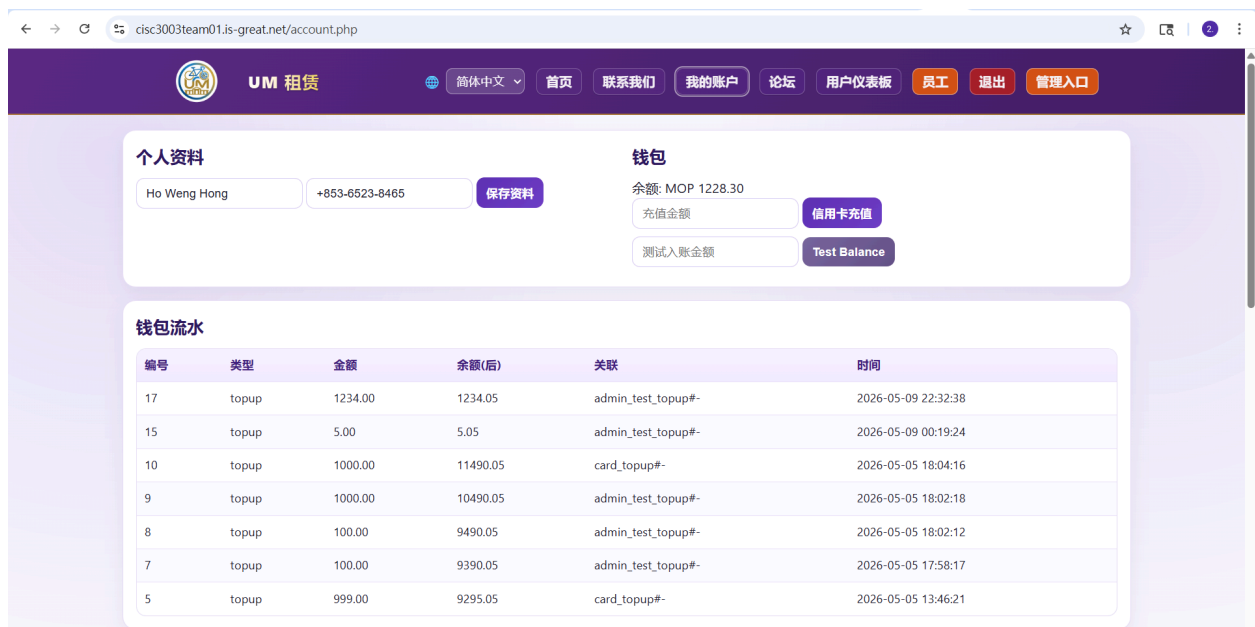


Figure 5. Wallet and Account Management

Shows student account details, current wallet balance, and transaction history table.

14.3 Staff and Admin Operations

Staff Navigation

- Bicycle Management
- Student Account Management
- Feedback Management
- Forum Management

Student Account Management

Add Student

Campus ID (e.g. s2000010) Full name Email
Phone (optional) Password (min 8 chars, letters) **Add Student**

Quick Notes

Remove means permanent delete for students without related records.
If the student has orders/posts/feedback, remove will be blocked.

Paste CSV:
campus_id,full_name,email,phone,password

Export Students CSV **Import Students CSV** active

Batch Update Selected

☐	ID	Campus ID	Name	Email	Status	Action
☐	35	g_longlei73_664	coco_lei73	longlei73@gmail.com	active	Update Remove
☐	34	g_tthh2555114_239	許許	tthh2555114@gmail.com	active	Update Remove
☐	33	SAM	SAM	dc32509@um.edu.mo	frozen	Update Remove
☐	32	g_th2555114_583	Thomas Hoi	th2555114@gmail.com	disabled	Update Remove
☐	31	g_a3253705196_198	Cristian	a3253705196@gmail.com	active	Update Remove

Figure 6. Staff Dashboard

Shows staff-side operational portal for fleet monitoring and student account management.

Admin Navigation

- Vehicle Management
- Staff Accounts
- Station Status
- Abnormal Orders
- Payment Pending
- Reports

Staff Account Management

staff001 Full Name
staff@um.edu.mo +853-0000-0000
Password **Add Staff**

ID	Campus ID	User	Email	Status	Action
11	t2000001	Dr. Maria Wong	maria.wong@um.edu.mo	active	Update Status Remove

UM Rental

About UM Mobility

- Home
- About the system
- Services
- Contact

Portals & access

- Login
- Register
- Staff portal
- Admin portal

Terms & policies

- Terms of service
- Privacy policy
- Cookie notice

Payment methods

Wallet & campus payments (demo badges)

VISA Mastercard AMEX
JCB LINE Pay Apple Pay
JKO Pay AFTEE Google Pay

Figure 7. Admin Portal

Shows top-level admin controls including staff account management and manual operational override (e.g., force-close).

14.4 Responsive Web Design (RWD)

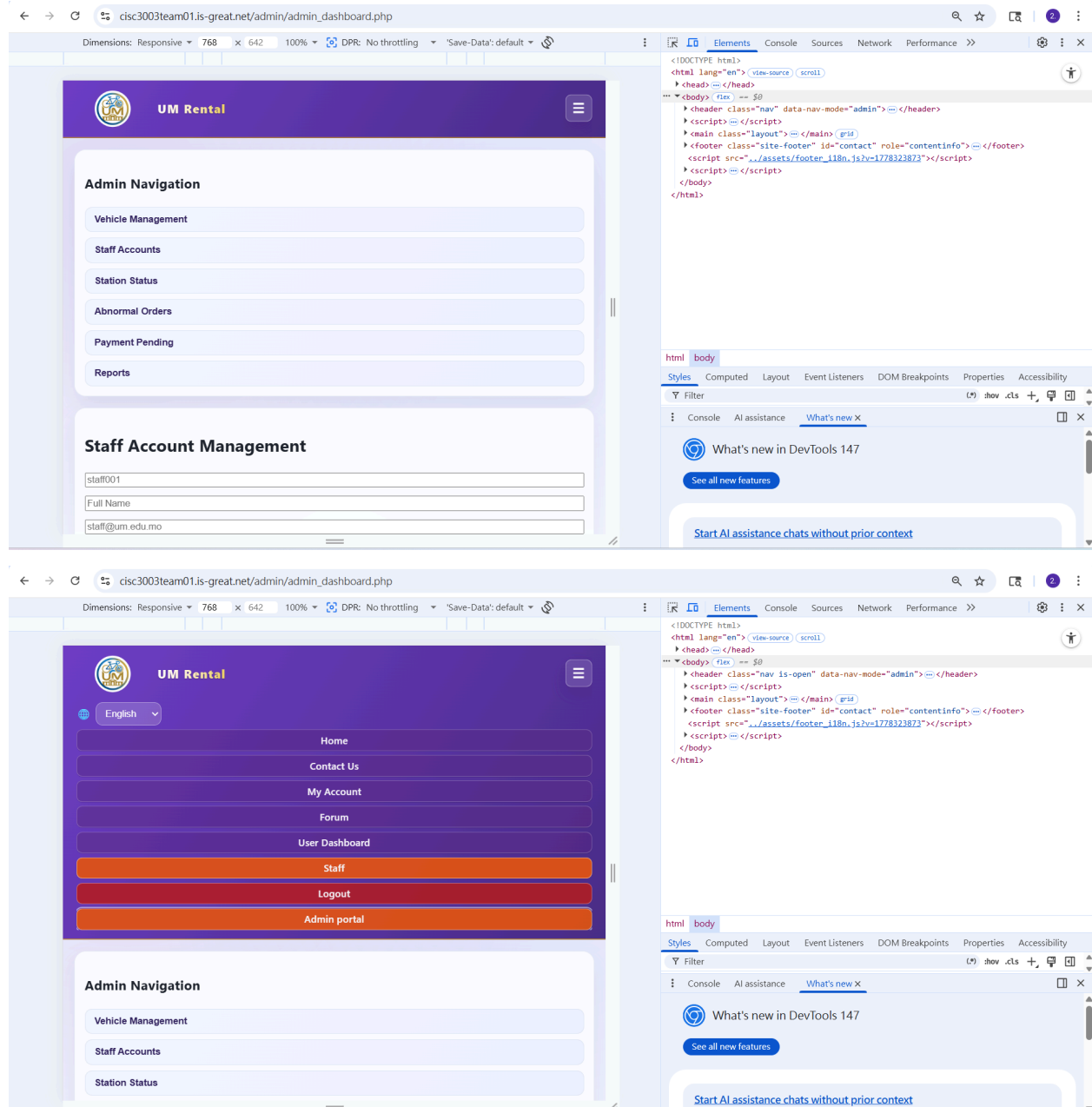


Figure 8. Mobile Responsive Views

Shows landing page and student dashboard in mobile viewport to demonstrate responsive layout behavior.

15. Project Citations and References

InfinityFree. (n.d.). *Free web hosting with PHP and MySQL*. <https://www.infinityfree.com/>

Leaflet. (n.d.). *Leaflet: An open-source JavaScript library for mobile-friendly interactive maps*. <https://leafletjs.com/>

PhpMyAdmin developers. (n.d.). *phpMyAdmin documentation*. <https://docs.phpmyadmin.net/>

The PHP Group. (2026). *PHP: Hypertext Preprocessor*. <https://www.php.net/>

Apache Friends. (2026). *XAMPP installers and downloads for Apache Friends*. <https://www.apachefriends.org/>

Oracle. (2026). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/>

Docker, Inc. (2026). *Docker documentation*. <https://docs.docker.com/>

Google Cloud. (2026). *Cloud Run documentation*. <https://cloud.google.com/run/docs>

Google Cloud. (2026). *Cloud SQL documentation*. <https://cloud.google.com/sql/docs>

OWASP Foundation. (n.d.). *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>

MDN Web Docs. (n.d.). *JavaScript*. <https://developer.mozilla.org/docs/Web/JavaScript>

GitHub, Inc. (2026). *GitHub documentation*. <https://docs.github.com/>

W3C. (n.d.). *Responsive web design basics / web standards resources*. <https://www.w3.org/>

16. Declaration of AI Usage

Artificial Intelligence (AI) tools (including Google Gemini and similar generative assistants) were used in this project under controlled and transparent conditions:

1. Code Formatting and Refactoring

AI was used to assist with CSS organization, normalization of PHP JSON response structures, and drafting boilerplate HTML for dashboard components.

2. Debugging Assistance

AI supported troubleshooting of the Unexpected token '<' issue by helping identify cases where PHP warnings/errors were returned as HTML instead of JSON during rate-limiting database failures.

3. Documentation and Report Structuring

AI was used to help summarize development notes/logs and transform them into structured academic writing for proposal/report preparation.

4. Localization Support (i18n)

AI assisted in drafting and validating key-value translations for Simplified Chinese (zh-CN) and Traditional Chinese (zh-TW) interface content.

All final implementation decisions, core business logic, database schema design, system architecture, testing judgments, and submission validation were performed and approved by the human project team.

17. Installation & Deployment Instructions

This project supports both local development/testing and live hosted deployment.

17.1 Local Development and Testing (XAMPP)

1. Install XAMPP (Apache + MySQL + PHP 8+).
2. Copy the project folder into *htdocs*.
3. Start Apache and MySQL in the XAMPP Control Panel.
4. Create the project database in phpMyAdmin.
5. Import schema and data scripts in sequence:
 - base schema / migration scripts
 - *basic_seed.sql* (general development) or *seed_demo_data.sql* (demo/grading)
6. Run *setup_database.bat* if provided.
7. Open the project in browser <http://localhost/project> and verify login, dashboard, rental flow, and wallet actions.

17.2 Live Deployment (InfinityFree)

1. Upload project files to the InfinityFree hosting environment.
2. Configure runtime settings and database connection credentials according to host requirements.
3. Create/import MySQL database through InfinityFree's built-in phpMyAdmin.
4. Validate key pages and API flows after deployment.

Live URL: <https://cisc3003team01.is-great.net/>

17.3 Demo Reset Procedure

To restore a clean grading/demo state, execute *reset_demo_state.sql*, then re-import *seed_demo_data.sql*.

17.4 Container and Cloud Readiness

The system includes Docker deployment artifacts (Dockerfile, *.dockerignore*) and is prepared for future migration to Google Cloud Run + Cloud SQL.

18. Incomplete Items (Not completed by deadline)

1. Production migration to Google Cloud Run + Cloud SQL is prepared but not finalized as the active deployment target at submission time.
2. Automated CI/CD pipeline integration for one-click deployment is planned but not fully completed in this submission.

19. Remarks and Academic Integrity Statement

This report is originally written based on the team's own project design, implementation process, testing outcomes, and reflection. No unattributed copy-and-paste content from published sources has been used. All external references, frameworks, and technical resources are properly cited in the References section using APA style.

The team acknowledges and complies with UM rules on academic honesty and proper attribution. Any AI-assisted support used during development or documentation is transparently declared in the "Declaration of AI Usage" section, and all final technical decisions, code validation, and submission responsibility remain with the student team.